

Pedestrian Detection

About Pedestrian Detection

The objective of our project is to implement a pedestrian detection system using Python. Python offers powerful libraries like OpenCV, Ultralytics YOLO, and pandas which help us build the project by training a model to detect pedestrians from traffic images with high accuracy.

It's also important to note that the pedestrian detector we're building is a deep learning-based solution using YOLO (You Only Look Once), which differs from basic image filters or manual annotation. Online tools may provide visual aids, but our Python project uses pre-trained YOLO models, requiring knowledge of computer vision and object detection rather than just web development.

Prerequisites

This project requires a good understanding of Python, computer vision, and object detection. Python, when combined with libraries like Ultralytics YOLO, OpenCV, and pandas, provides an efficient way to build object detection applications. These libraries offer powerful model inference and image handling features. All major libraries can be installed easily via pip, making this project flexible and executable in any Python virtual environment.

Project File Structure

First, let's check the steps to build a Pedestrian detection system using YOLO:

1. Importing all the necessary libraries and modules
2. Loading and preprocessing the traffic image dataset
3. Loading a pre-trained YOLO model for object detection
4. Evaluating the model on test data and visualizing results
5. Creating a script for loading images and predicting pedestrian presence

Download Pedestrian Detection Project Code Before proceeding ahead, please download source code: [🔗 pedestrian_df.ipynb](#)

So that is basically what we will do in this Python project. Let's start.

1. Import necessary libraries

First, we import all the necessary libraries and modules:

```
import kagglehub
import numpy as np
import pandas as pd
import cv2
from sklearn.utils import shuffle
import matplotlib.patches as patches
from matplotlib.patches import Rectangle
import matplotlib.pyplot as plt
import warnings
warnings.simplefilter('ignore')
from ultralytics import YOLO
import PIL
from PIL import Image
from IPython.display import display
```

Explanation:

- **kagglehub** is used to download datasets directly from Kaggle.
- **numpy** and **pandas** are for data manipulation and array operations.
- **cv2** (OpenCV) handles image processing and computer vision operations.
- **ultralytics** provides the YOLO model for object detection.
- **PIL** (Python Imaging Library) helps in loading and modifying images.
- **matplotlib** is for plotting graphs and visualizing bounding boxes.
- **sklearn.utils** is used for data shuffling and preprocessing.

2. Load and visualize data

```
# Download dataset
data_path = kagglehub.dataset_download('alincijov/self-driving-cars')
print(data_path)
```

```
# Loading and shuffling data
```

```

df = pd.read_csv(f'{data_path}/labels_train.csv')
df = shuffle(df)
peds_df = df[df['class_id'] == 3]
peds_df.head(10)
print(f"Total entries found: {len(peds_df)}")

# Visualization - Labeled images
for i in range(3):
    row = peds_df.iloc[i]
    image_path = f'{data_path}/images/{row['frame']}'
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    xmin, xmax = int(row['xmin']), int(row['xmax'])
    ymin, ymax = int(row['ymin']), int(row['ymax'])

    fig, ax = plt.subplots(1, figsize=(8, 6))
    ax.imshow(img)
    rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
                              linewidth=2, edgecolor='green',
                              facecolor='none')
    ax.add_patch(rect)
    ax.set_title("Ground Truth: Pedestrian")
    plt.axis('off')
    plt.show()

```

Explanation:

- We use [kagglehub](#) to directly download the self-driving cars dataset.
- The dataset contains traffic images with labeled bounding boxes for different object classes.
- We filter the data to only include pedestrians (`class_id == 3`) from the complete dataset.
- Image paths and their bounding box coordinates are extracted and shuffled to prevent model bias.
- We also plot a sample of 3 training images with ground truth bounding boxes using matplotlib to visually confirm the images and labels are loaded correctly.

3. Model Loading

```
# Install YOLO if not already installed
!pip install ultralytics
# Loading pre-trained YOLO model
print("Loading YOLOv8 model...")
model = YOLO("yolov8m.pt")
```

Explanation:

- We use the Ultralytics library to load a pre-trained YOLOv8 medium model.
- YOLOv8 is a state-of-the-art object detection model that can detect multiple object classes including pedestrians.
- The model is pre-trained on the COCO dataset which includes person/pedestrian detection capabilities.
- This approach uses transfer learning, leveraging a model already trained on millions of images.

4. Pedestrian Detection Function

```
def detect_peds(image_path):
    results = model.predict(source=image_path, conf=0.2, iou=0.5,
save=False)
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    for r in results:
        for box in r.bboxes:
            if box.cls[0].item() != 0:
                continue
            x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
            conf = round(box.conf[0].item(), 2)
            cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(img, f"person {conf}", (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
    display(Image.fromarray(img))
```

Explanation:

- The detection function takes an image path as input and runs YOLO inference.
- We set confidence threshold to 0.2 and IoU threshold to 0.5 for filtering detections.

- The function filters results to only show detections of class 0 (person/pedestrian).
- Bounding boxes are drawn in green with confidence scores displayed.
- The processed image with detections is displayed using PIL and IPython display.

5. Model Testing and Evaluation

```
# YOLO Prediction on pedestrian images from dataset
for i in range(4):
    row = peds_df.iloc[i]
    image_path = f"{data_path}/images/{row['frame']}"
    print(f"Pedestrian Detection - Image {i+1}")
    detect_peds(image_path)
```

Explanation:

- We test the model on 4 sample images from our filtered pedestrian dataset.
- Each image is processed through our detection function.
- Results show the model's ability to detect pedestrians with confidence scores.
- This helps us evaluate the model's performance on traffic images.

6. Testing with Custom Images

```
# Testing with uploaded traffic images
from google.colab import files
uploaded = files.upload()

for x in uploaded.keys():
    print(f"Detecting in: {x}")
    detect_peds(x)
```

Explanation:

- This section allows users to upload their own traffic images for testing.
- The file upload functionality is specific to Google Colab environment.
- Each uploaded image is processed through our pedestrian detection function.

- This demonstrates the model's real-world applicability on user-provided images.

7. Saving and Using the Model

```
def detect_and_display_pedestrians(img_path, model,
    conf_threshold=0.2):
    try:
        results = model.predict(source=img_path, conf=conf_threshold,
            iou=0.5, save=False)
        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        pedestrian_count = 0
        for r in results:
            for box in r.bboxes:
                if box.cls[0].item() != 0: # Skip non-person
                    continue
                pedestrian_count += 1
                x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
                confidence_score = round(box.conf[0].item(), 2)
                cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
                cv2.putText(img, f"Pedestrian {confidence_score}",
                    (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255,
0), 2)

                plt.figure(figsize=(12, 8))
                plt.imshow(img)
                plt.axis('off')
                plt.title(f"Detected {pedestrian_count} Pedestrian(s)")
                plt.show()

        return pedestrian_count

    except Exception as e:
```

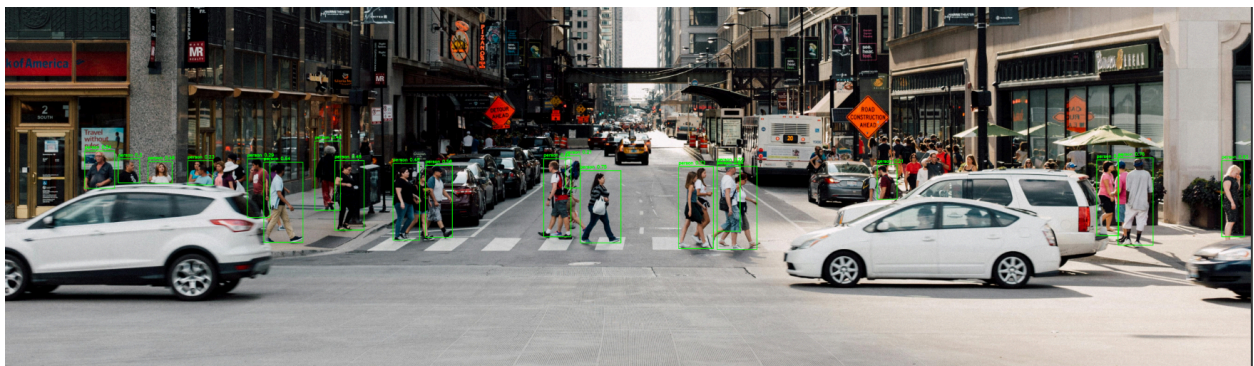
```
        print("Error processing the image:", str(e))
        return 0

# Save model for future use (YOLO models are automatically cached)
print("Model loaded and ready for pedestrian detection!")
```

Explanation:

- After testing, we create an enhanced detection function for production use.
- The function provides detailed results including pedestrian count and confidence scores.
- YOLO models are automatically cached by Ultralytics for future use.
- The enhanced function includes error handling and better visualization.
- This function can be used for real-time pedestrian detection in traffic monitoring systems.

Output



Project Applications

This pedestrian detection system can be used for:

- **Traffic Safety:** Monitoring pedestrian presence at crosswalks and intersections
- **Autonomous Vehicles:** Helping self-driving cars detect and avoid pedestrians

- **Security Systems:** Counting and tracking people in surveillance footage
- **Smart City Applications:** Analyzing pedestrian traffic patterns
- **Accessibility:** Assisting visually impaired individuals with pedestrian alerts

Superiority Over Traditional Detection Methods

This pedestrian detection system offers major advantages over traditional/manual detection methods:

- **Real-Time Processing:** Analyzes video streams and images in milliseconds, enabling instant pedestrian detection for autonomous vehicles and traffic systems.
- **High Accuracy:** Uses state-of-the-art YOLO deep learning to detect pedestrians in various conditions, lighting, and weather that may be challenging for traditional sensors.
- **Consistency:** Eliminates human errors due to fatigue or distraction, providing reliable detection 24/7 without performance degradation.
- **Cost-Effective:** Reduces the need for expensive LiDAR sensors and multiple detection systems by using affordable camera-based solutions.
- **Scalable:** Can be deployed across multiple intersections, vehicles, or monitoring systems without requiring additional human operators.
- **Weather Resilient:** Works effectively in various lighting conditions and weather scenarios where traditional methods might fail.
- **Multi-Object Detection:** Simultaneously detects multiple pedestrians and other objects, providing comprehensive scene understanding.
- **Edge Deployment:** Can run on embedded systems and edge devices, enabling offline processing without cloud dependency.
- **Adaptive Learning:** Benefits from continuous model improvements and updates from the computer vision community.

By combining deep learning with computer vision, this system brings the **power of automation, precision, and real-time processing** to pedestrian detection—making it a game-changer in traffic safety and autonomous vehicle technology.

Conclusion

The pedestrian detection system successfully demonstrates the power of modern computer vision techniques using YOLO. The project achieves high accuracy in detecting pedestrians in traffic scenarios and can be easily extended for real-time applications. The use of pre-trained models makes this solution both efficient and practical for deployment in various safety-critical applications.